Komunikace s databázovým systémem

Databáze

RDBMS (Relation DataBase Management System)

Není to jen uložiště dat, je to sofistikovaný nástroj, který řeší sousty problémů a zároveň je jednoduchý k použití.

Řeší třeba problém současné editace stejné položky několika uživateli ve stejný okamžik, který by jinak mohl zapříčinit nekonzistenci databáze. RDBMS data v tomto případě zamkne a odemkne až po vykonání zápisu.

Dále umožňuje spojovat několik dotazů do transakcí, kdy se série dotazů vykoná vždy celá nebo vůbec.

Tyto vlastnosti databázového stroje jsou shrnovány zkratkou ACID

ACID - Atomicity (nedělitelnost) Consistency (validita), Isolation (izolace) a Durability (odolnost)

Atomicity- transakce se provede celá nebo vůbec. Provádí se jako jedna atomická operace.

= pokud nějaká část operace selže, vrátí se databáze do původního stavu a žádné části transakce nebudou provedeny.

Třeba tohle je důležité v bankovní databázi, když se peníze nepodaří odečíst z prvního účtu tak se nepřipíší na účet druhý

Consistency- Stav databáze po dokončení transakce je vždy konzistentní, tedy validní podle všech definovaných pravidel a omezení. Nikdy nenastane situace, že by se databáze nacházela v nekonzistentním stavu.

Isolation - Operace jsou izolované a navzájem se neovlivňují. Pokud se sejde v jeden okamžik více dotazů na zápis do stejného řádku, jsou vykonávány postupně, jako ve frontě. FIFO

Durability - Všechna zapsaná data jsou okamžitě zapsána na trvanlivá úložiště (na pevný disk), v případě výpadku el. energie nebo jiného přerušení provozu RDBMS vše zůstane tak, jak bylo těsně před výpadkem

<u>S databází se komunikuje jazykem SQL</u>

3 základní hotové databáze co umíme

- MSSQL – Microsoft SQL
- MySQL
- Oracle

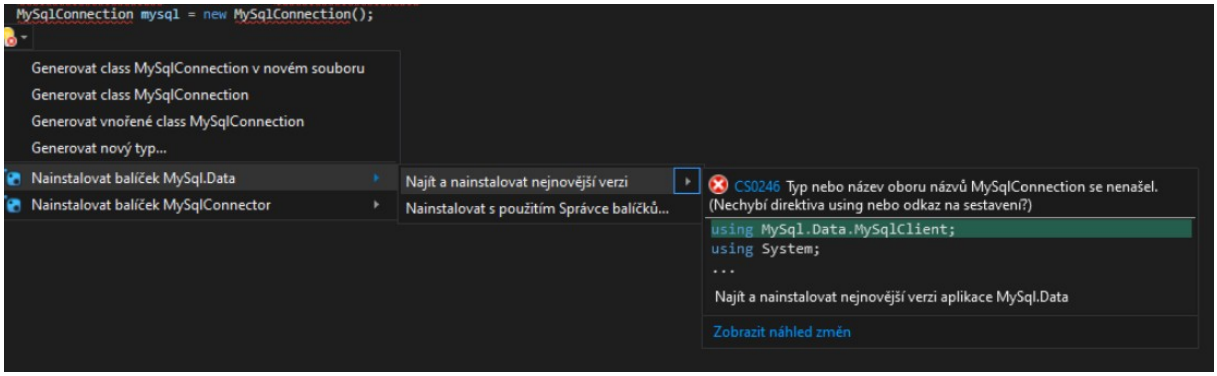
ADO.NET je knihovna, která zastřešuje technologie pro přístup k datům a práci s nimi. Je součástí .NET frameworku.

- Poskytovatelé dat ADO.NET:
  - SqlClient (`System.Data.SqlClient`)
  - OLEDB (`System.Data.OleDb`)
  - ODBC (`System.Data.Odbc`)
  - OracleClient (`System.Data.OracleClient`)

Visual Studio má nástroje optimalizované pro MSSQL, protože je od Microsoftu stejně jako C# a Visual Studio, nástroje pro Oracle má visual studio taky, protože microsoft s ním počítá, často je určena pro velké podniky.

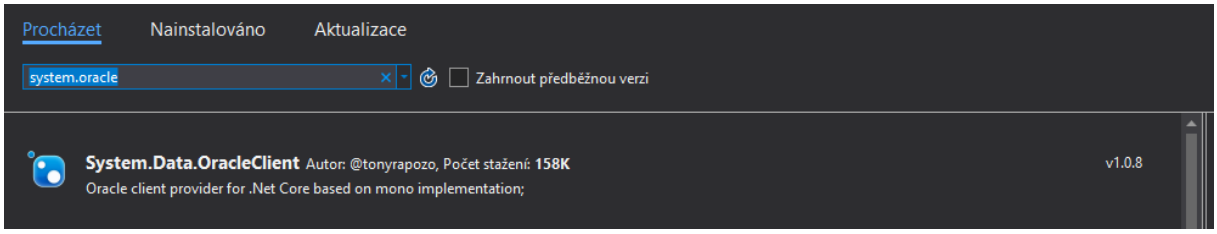
Nástroje pro ostatní databáze jsou zastaralé a jejich vývoj dávno skončil.

MySQL - MySql.Data.MySqlClient

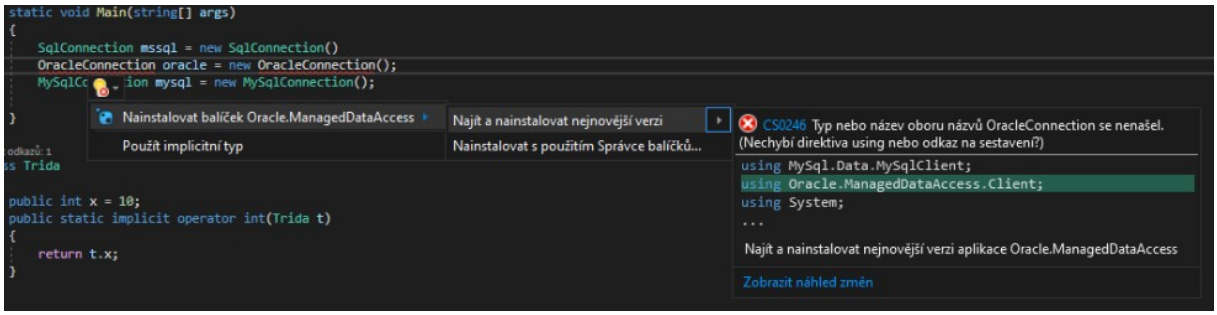


Oracle – System.Data.OracleClient (ADO.NET) nebo Oracle.ManagedDataAccess.Client (ODP.NET)

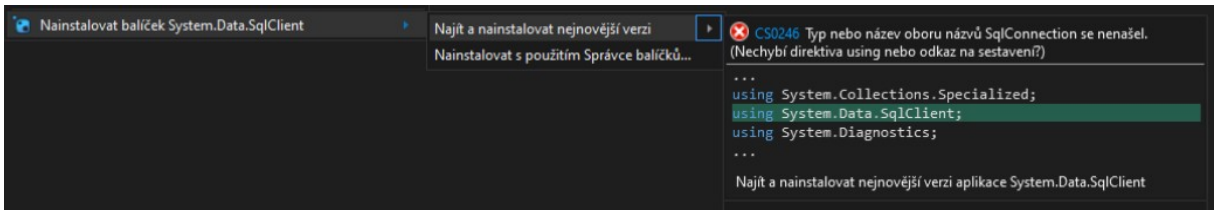
ODP.NET je novější a modernější



(musí se do c# přidat, i když je ADO.NET, tak jako prioritní using je nabídnu ODP.NET)



MSSQL – System.Data.SqlClient



Rozpor objektového a relačního přístupu

Relační databáze jsou ověřený způsob jak pracovat s daty. I když existují i databáze plně objektové, tak moc firem je nepoužívá. Např: MongoDB

<u>Revoluce v programování – příchod objektů</u>

= problém s ukládáním dat, jelikož relační databáze objektově nefungují a objekty ukládat neumí.

Databázový Wrapper

umožňuje s databází pracovat jako s objektem, nicméně komunikujeme s ní stále v jejím jazyce SQL.

Míchá se objektový a relační kód.

Výhodou je zachování výkonu

Objektově relační mapování

Z databáze místo pole hodnot se dostanou rovnou objekty a ty na sobě mají metody. SQL jazyk se nepoužívá v programu, tabulky v databázi vidíme jako kolekce objektů, se kterými můžeme pracovat běžnými prostředky jazyka.

Nevýhody: a pozadí dochází k velké degradaci výkonu databáze, SQL dotazy se generují automaticky a jsou často neefektivní. Je těžké naprogramovat

Objektové databáze

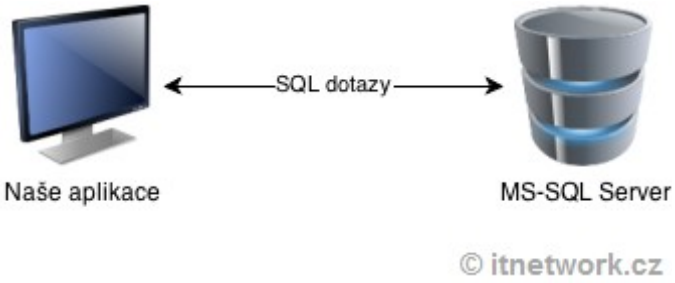
Není moc používaná, drahá atd..

Připojení do databáze

Připojená aplikace

Když se potřebujeme načíst, vkládat, měnit nebo mazat data v reálném čase

DataReader, Command, Connection obsahují SQL příkazy a dostávají výsledky, jako kdyby se SQL příkazy psaly v databázovém programu.



Odpojená aplikace – DataSet

<u>Jedná se o Databázový Wrapper</u>

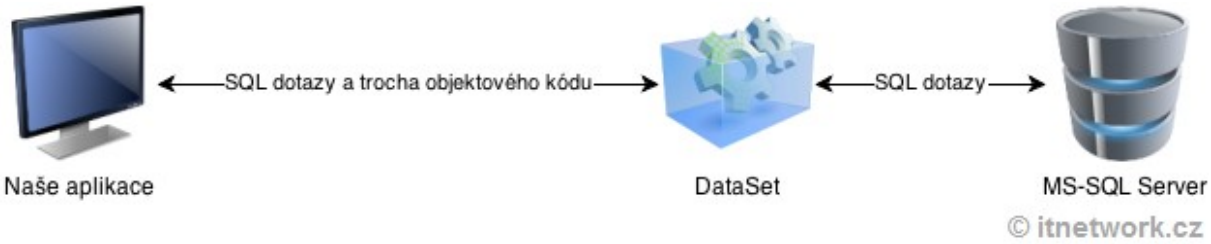
V operační paměti je DataSet, který v sobě má data se kterýma aplikace pracuje.

Občas se DataSet sesynchronituje s databází.

DataSet v sobě obsahuje tabulky (DataTable), tabulka řádky(DataRow) a řádek sloupce (DataColumn).

Tabulka je objekt. Můžou se do ní řádky přidávat a upravovat je bez psaní SQL kódu.

Když chceme spustit na databázi nějaký příkaz, použijeme DataAdapter, pomocí kterého si naplníme DataSet daty. Příkazy již musíme psát v jazyce SQL dané databáze.



LINQ To SQL

<u>Jedná se o objektově relační mapování</u>

LINQ to SQL poskytuje kompletní objektovou abstrakci nad databází

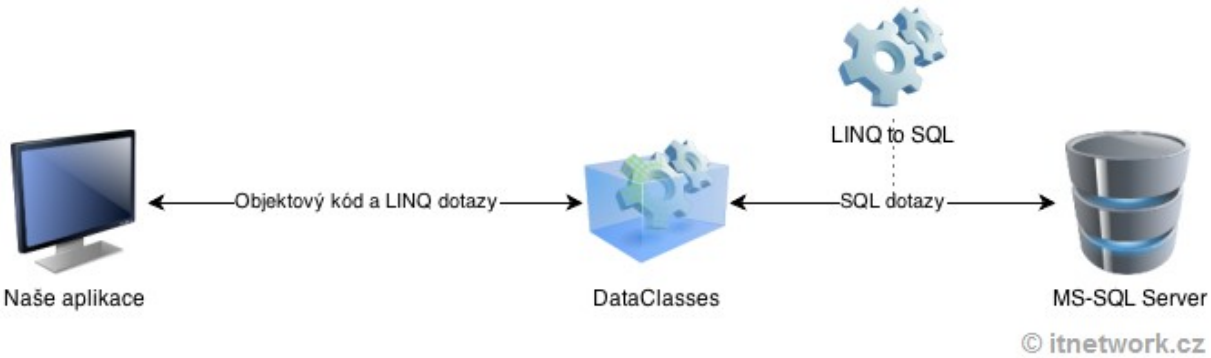
Databázi v programu vidíme jako List Objektů

Nevíme o tabulkách, od příkazů dostáváme rovnout objekty

Vše se děje na pozadí automaticky

Pracujeme s DataClasses, objektrová struktura databáze, obsahuje třídy pro jednotlivé tabulky, sloupce tabulky jsou vlastnosti daných tříd.

pouze s DataClasses, ta potom na pozadí pomocí LINQ to SQL komunikuje s databází a provádí za nás SQL dotazy.



Nevýhoda, horší výkon

Entity Framework

Pokročilejší LINQ TO SQL,

Umí vazbu M:N

Ještě horší výkon

Připojení z C# do DB

Abychom se mohli k DB připojit, potřebujeme tzv. ConnectionString. To je řetězec, který obsahuje údaje potřebné k připojení k databázi. Typicky je zde název databáze a případně heslo.

Píše se do závorek deklarace konekce nebo jako její vlastnost u všech 3.

MSSQL

DataSource = adresa db serveru

InitialCatalog = název db

Password =heslo

UserID = username

MYSQL

Server/Host/Data Source/Data Source/Address = adresa db serveru

Database /Initial Catalog = název db

Password/pwd =heslo

Username/User Id/Uid = username

```
MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; Database=maturita_jirka; User ID=jirka;password=heslo");
```

Nebo

```
MySqlConnection mysql = new MySqlConnection();
mysql.ConnectionString = "Data Source=debianserver.cz; Database=maturita_jirka; User ID=jirka;password=heslo";
```

Nebo

SQLConnectionStringBuilder ten má jako vlastnosti Username,Password atd…

Je dobrým zvykem všechny databázové objekty tvořit v bloku using, aby došlo k rychlému zavření nepotřebných spojení.

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; Database=maturita_jirka; User ID=jirka;password=ZNnc4NKB"))
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        Console.WriteLine("OPEN");
    }
}
```

<u>ConnectionState je v System.Data</u>

Většina tříd je stejná jen podle databáze jinak zaříná jejich název OracleCommand, SqlCommand a MySqlCommand

Databázové příkazy

Třída SqlCommand nebo MySqlCommand, k dispozici je taky třída builder jako u connection stringu.

Píše se do ní příkazy v jazyce SQL.

Důležité vlastnosti jsou

Connection = instance connection (v jaké db se má command provést)

CommandText = příkaz v jazyce SQL

```
MySqlCommand c = new MySqlCommand();
c.
```

▲ 3 z 4 ▼ MySqlCommand(string **cmdText**, MySqlConnection connection)
Initializes a new instance of the MySqlCommand class with the text of the query and a MySqlConnection.
cmdText: *The text of the query.*

<u>Často se zapomíná na connection</u>

Metody na spuštení

ExecuteReader() - Tuto první využijeme tehdy, pokud SQL dotaz vrací množinu dat (většina příkazů SELECT, např. dotaz na jedno nebo více slovíček).

ExecuteScalar() - Druhá metoda nalezne své uplatnění, pokud SQL dotaz vrací jedinou hodnotu (typicky SELECT s agregační funkcí, např. počet všech slovíček).

ExecuteNonQuery() - Poslední metoda se používá ke spouštění dotazů, které nevracejí žádnou hodnotu (příkazy INSERT, UPDATE, DELETE, např. vložení slovíčka). Její návratová hodnota nese informaci o počtu "ovlivněných" řádků.

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; Datab
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlCommand c = new MySqlCommand("SHOW TABLES", mysql);
        MySqlDataReader a = c.ExecuteReader();
        while (a.Read())
        {
            Console.WriteLine(a.GetString(0));
        }
    }
}
```

<u>ExecuteReader() a ExecuteScalar() vrací MySqlDataReader (pro mysql)</u>

Ta umožňuje procházet výslednou množinu dat po jednotlivých záznamech.

Metoda Read(), ta se dává do while cyklu (dokud bude mít řádky ke čtění), protože nevíme kolik záznamů je v tabulce

Pro přístup k hodnotě na řádku se používá **r.GetInt32(index)** (převede na požadovaný datový typ) nebo **r[“nazev sloupce“]** nebo **r[index]**

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.c
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlCommand c = new MySqlCommand("select id from uzivatel", mysql);
        MySqlDataReader r = c.ExecuteReader();
        while (r.Read())
        {
            Console.WriteLine(r["id"]);
        }
    }
}
```

Zadání textu do commandu, není dobrý dělat řetězec pomocí string+string

Ani string + nejakejObject.Jmeno.ToString()

Protože když třeba to nejakejObject.Jmeno nastavím na “DROP DATABASE neco“ tak autor programu umožnil smazat svým programem svoji databázi.

<u>To se nazývá SQL injection</u>

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; Database=maturita_jirka;
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        string username = "moshpit";
        MySqlCommand c = new MySqlCommand("select id from uzivatel where username = @Username", mysql);
        c.Parameters.AddWithValue("@Username",username);
        MySqlDataReader r = c.ExecuteReader();
        while (r.Read())
        {
            Console.WriteLine(r.GetString(0));
        }
    }
}
```

<u>Ochrana pomocí Parameters</u>

AddWithValue(“@proměná v sql commandu“,zdroj naší promění co chce vložit, nebo její obsah)

```
c.Parameters.Add()
▲ 5 z 5 ▼ MySqlParameter MySqlParameterCollection.Add(string parameterName, MySqlDbType dbType, int size, string sourceColumn)
Adds a MySqlParameter to the MySqlParameterCollection with the parameter name, the data type, the column length, and the source column name.
parameterName: The name of the parameter.
```

AddValue je použit třeba u DataGridView ve Forms

<u>DB si sama parametry ošetří, takže se tam škodolibý příkaz nedostane</u>

Když v máme v DB třeba proměnou where, tak by kolidovala s podmínkovým příkazem, proto se proměný píšou do hranatých závorek

```
MySqlCommand c = new MySqlCommand("select `id` from `uzivatel` where `username` = @Username", mysql);
```

V MySQL je to Grave accent ` ← tohle místo hranatých závorek

DataSet a DataAdapter

DataSet umožňuje uchovávat a upravovat velké množství dat v lokální paměti bez stálého připojení ke zdroji dat, proto je odpojená aplikace.

DataAdapter slouží jako most mezi DataSet a SQL Server pro načítání a ukládání dat.

Obsahuje kolekci komponent DataTable (ve vlastnosti Tables), které reprezentují jednotlivé databázové tabulky (DataTable).

Navíc může obsahovat informace o vztazích mezi těmito tabulkami.

Často je DataAdapter pouze pro jednu tabulku.

Poskytuje tento most podle mapování Fill , který změní data v, DataSet aby odpovídala datům ve zdroji dat, a Update který změní data ve zdroji dat tak, aby se shodovala s daty v, a to DataSet pomocí příslušných příkazů jazyka Transact-SQL pro zdroj dat. Aktualizace se provádí po jednotlivých řádcích. U každého vloženého, změněného a odstraněného řádku Update Metoda určuje typ změny, která byla provedena v této metodě (Insert , Update nebo Delete)

V DataSet paměti, která je mezipaměť dat načtená ze zdroje dat, je hlavní součástí architektury ADO.NET. DataSetObsahuje kolekci DataTable objektů, které lze vzájemně propojit s DataRelation objekty. Integritu dat můžete také vymáhat DataSet pomocí UniqueConstraint ForeignKeyConstraint objektů

Vypsání názvů sloupců v tabulce uživatel v DataTable

```
DataSet ds = new DataSet();
DataTable dt = new DataTable();
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.c
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlCommand c = new MySqlCommand("select * from `uzivatel`", mysql);
        MySqlDataAdapter a = new MySqlDataAdapter();
        a.SelectCommand = c;
        a.Fill(ds);
        ds.Tables[0].TableName = "uzivatel";
    }
}
dt = ds.Tables["uzivatel"];
foreach (DataColumn item in dt.Columns)
{
    Console.WriteLine(item);
}
```

Jméno tabulky typu DataTable v DataSetu se dá přidat v metodě Adapteru Fill()

```
a.Fill(ds,"uzivatel");
```

Když se tabulka nepojmenuje tak má jméno “Table“, je důležité pojmenovávat tabulky, jinak se všechno ukládá do jedný

Vypsání celých řádků v tabulce uživatel v DataTable

```
DataSet ds = new DataSet();
DataTable dt = new DataTable();
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.c
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlCommand c = new MySqlCommand("select * from `uzivatel`", mysql);
        MySqlDataAdapter a = new MySqlDataAdapter();
        a.SelectCommand = c;
        a.Fill(ds);
        ds.Tables[0].TableName = "uzivatel";
    }
}
dt = ds.Tables["uzivatel"];
foreach (DataRow item in dt.Rows)
{
    foreach (var x in item.ItemArray)
    {
        Console.Write(x+", ");
    }
    Console.WriteLine();
}
```

Vypsání určitých sloupců v tabulce uživatel v DataTable

```
DataSet ds = new DataSet();
DataTable dt = new DataTable();
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz;
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlCommand c = new MySqlCommand("select * from `uzivatel`", mysql);
        MySqlDataAdapter a = new MySqlDataAdapter();
        a.SelectCommand = c;
        a.Fill(ds);
        ds.Tables[0].TableName = "uzivatel";
    }
}
dt = ds.Tables["uzivatel"];
foreach (DataRow item in dt.Rows)
{
    Console.WriteLine($"{item["id"]} {item["username"]} {item["password"]}");
}
```

Vypsání určitých sloupců v tabulce uživatel bez DataTable

```
DataSet ds = new DataSet();
  DataTable dt = new DataTable();
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlCommand c = new MySqlCommand("select * from `uzivatel`", mysql);
        MySqlDataAdapter a = new MySqlDataAdapter();
        a.SelectCommand = c;
        a.Fill(ds);
        ds.Tables[0].TableName = "uzivatel";
    }
}
foreach (DataRow item in ds.Tables["uzivatel"].Rows)
{
    Console.WriteLine($"{item["id"]} {item["username"]} {item["password"]}");
}
```

Adapter do svého konstruktoru dokáže vytvořit třídu Command ze String a Connection co mu dáme

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; Database=maturita_jirka;
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlDataAdapter a = new MySqlDataAdapter("select * from `uzivatel` where username = @Id",mysql);
        a.SelectCommand.Parameters.AddWithValue("@Id", "moshpit");
        a.Fill(ds);
        ds.Tables[0].TableName = "uzivatel";
    }
}
foreach (DataRow item in ds.Tables["uzivatel"].Rows)
{
    Console.WriteLine($"{item["id"]} {item["username"]} {item["password"]}");
}
```

Tato metoda se nejčastěji volá s jedním parametrem - filtr (podmínka)

nebo se dvěma parametry (filtr, řazení dat).

Tento způsob je výhodný právě v konzolové aplikaci.

Select z tabulky, kterou máme v DataSet

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; D
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlDataAdapter a = new MySqlDataAdapter("select * from `uzivatel`",mysql
        a.Fill(ds,"uzivatel");

    }
}
foreach (DataRow item in ds.Tables["uzivatel"].Select("id=3"))
{
    Console.WriteLine($"{item["id"]} {item["username"]} {item["password"]}");
}
```

DataView je třída, která umožňuje různé pohledy na data v komponentě DataTable.

Mezi její základní vlastnosti patří RowFilter, Sort a RowStateFilter.

Základní pohled na tabulku získáme z vlastnosti DefaultView třídy DataTable.

Tento způsob se více využívá ve formulářových aplikacích, neboť DataView můžeme použít přímo jako datový zdroj (DataSource) pro komponentu DataGridView.

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; Data
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlDataAdapter a = new MySqlDataAdapter("select * from `uzivatel`",mysql);
        a.Fill(ds,"uzivatel");

    }
}
DataView v = new DataView(ds.Tables["uzivatel"]);
v.Sort = "id";
foreach (DataRowView item in v)
{
    Console.WriteLine(item["id"]+" "+item["username"]);
}
```

Spojení tabulke v DataSetu

Tato relace se při přenosu dat do DataSet ztratí.

V C# je relace reprezentována třídou DataRelation

v DataSet jsou všechny relace uloženy v kolekci Relations.

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; Database=maturita_jirka; User ID=jirka;passwor
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlDataAdapter a = new MySqlDataAdapter("select * from `uzivatel`",mysql);
        a.Fill(ds,"uzivatel");
        MySqlDataAdapter a2 = new MySqlDataAdapter("select * from `drazba`", mysql);
        a2.Fill(ds, "drazba");

    }
}
DataRelation r = new DataRelation("UzivatelID",ds.Tables["drazba"].Columns["UzivatelID"],ds.Tables["uzivatel"].Columns["id"]);
ds.Relations.Add(r);
```

childColumn je tabulka ve které je foreign key

```
DataRelation r = new DataRelation("UzivatelID",ds.Tables["drazba"].Columns["UzivatelID"],ds.Tables["uzivatel"].Columns["id"]);
ds.Relations.Add(
foreach (var item
{
```

▲ 1 z 6 ▼ DataRelation(string relationName, DataColumn parentColumn, DataColumn childColumn)
Initializes a new instance of the DataRelation class using the specified DataRelation name, and parent and child DataColumn objects.
parentColumn: The parent DataColumn in the relationship.

Relaci přidáváme do DataSet jen tehdy, pokud ji plánujeme nějak využít. Typickým příkladem je využití metod GetChildRows a GetParentRow třídy DataRow, které s pomocí relace dokáží získat odpovídající data z vázané tabulky.

```
using (MySqlConnection mysql = new MySqlConnection("Data Source=debianserver.cz; Database=maturita_jirka; User ID=jirka;password
{
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlDataAdapter a = new MySqlDataAdapter("select * from uzivatel",mysql);
        a.Fill(ds,"uzivatel");
        MySqlDataAdapter a2 = new MySqlDataAdapter("select * from drazba", mysql);
        a2.Fill(ds, "drazba");

    }
}
DataRelation r = new DataRelation("UzivatelID", ds.Tables["uzivatel"].Columns["id"],ds.Tables["drazba"].Columns["UzivatelID"]);
ds.Relations.Add(r);

foreach (DataRow dr in ds.Tables["drazba"].Rows)
{
    Console.WriteLine(dr["id"]);
    foreach (DataRow dr2 in dr.GetParentRows("UzivatelID"))
    {
        Console.WriteLine($"- {dr2["id"]} {dr2["username"]}");
    }
}
```

Vložení z DataSetu do DB

Vytřit DataRow v DataSetu. Vložit jí do DataSetu. Vytvořit Adapter, napsat InsertCommand.

Update() je používá pro insert,delete a update;

```
mysql.Open();
if (mysql.State == ConnectionState.Open)
{
    MySqlDataAdapter a = new MySqlDataAdapter("select * from obraz",mysql);
    a.Fill(ds,"obraz");
    DataRow r = ds.Tables["obraz"].NewRow();
    r["nazev"] = "Pan Obraz";
    r["platno"] = "Batist";
    r["cena"] = 165;
    r["vaha"] = 2.5;
    r["datum"] = DateTime.Now;
    ds.Tables["obraz"].Rows.Add(r);
    MySqlDataAdapter a2 = new MySqlDataAdapter();
    a2.InsertCommand = new MySqlCommand("insert into obraz (nazev,platno,cena,vaha,datum) values (@Nazev,@Platno,@Cena,@Vaha,@Datum)", mysql);
    a2.InsertCommand.Parameters.Add("@Nazev", MySqlDbType.VarChar, 50, "nazev");
    a2.InsertCommand.Parameters.Add("@Platno", MySqlDbType.VarChar, 50, "platno");
    a2.InsertCommand.Parameters.Add("@Cena", MySqlDbType.Int32, 0, "cena");
    a2.InsertCommand.Parameters.Add("@Vaha", MySqlDbType.Int32, 0, "vaha");
    a2.InsertCommand.Parameters.Add("@Datum", MySqlDbType.Date, 50, "datum");
    a2.Update(ds.Tables["obraz"]);
}
```

Mazání

Najít v DataSetu row kterou chcem smazat. Vytvořit Delete command. A Update

```
    mysql.Open();
    if (mysql.State == ConnectionState.Open)
    {
        MySqlDataAdapter a = new MySqlDataAdapter("select * from obraz",mysql);
        a.Fill(ds,"obraz");

        DataRow[] r = ds.Tables["obraz"].Select("nazev='Pan Obraz4'");
        if (r.Length > 0)
        {
            DataRow rr = r[0];
            rr.Delete();
        }
        MySqlDataAdapter a2 = new MySqlDataAdapter();
        a2.DeleteCommand = new MySqlCommand("delete from obraz where nazev = @Nazev", mysql);
        a2.DeleteCommand.Parameters.Add("@Nazev", MySqlDbType.VarChar, 50, "nazev");

        a2.Update(ds.Tables["obraz"]);
    }
}
```

Psaní commandů update, delete a insert by bylo moc dlouhé

MySqlCommandBuilder je umí vygenerovat, potřebuje k nim jenom select command, aby věděl jak se daná tabulka v DB jmenuje.

```
mysql.Open();
if (mysql.State == ConnectionState.Open)
{
    MySqlDataAdapter a = new MySqlDataAdapter("select * from obraz",mysql);
    a.Fill(ds,"obraz");

    DataRow[] r = ds.Tables["obraz"].Select("nazev='Pan Obraz3'");
    if (r.Length > 0)
    {
        DataRow rr = r[0];
        rr.Delete();
    }
    MySqlDataAdapter a2 = new MySqlDataAdapter();
    a2.SelectCommand = new MySqlCommand("select * from obraz", mysql);
    MySqlCommandBuilder b = new MySqlCommandBuilder(a2);
    a2.Update(ds.Tables["obraz"]);
}
```

Datové typy v C# na DB typy

Typ databázového stroje SQL Server	Typ rozhraní .NET Framework	Výčet SqlDbType	SqlDataReader SqlTypes typ přístupového objektu	Výčet DbType	Přístupový objekt typu SqlDataReader DbType
bigint	Int64	BigInt	GetSqlInt64	Int64	GetInt64
binary	Byte[]	VarBinary	GetSqlBinary	Binary	GetBytes
bit	Boolean	Bit	GetSqlBoolean	Boolean	GetBoolean
char	String Char[]	Char	GetSqlString	AnsiStringFixedLength, String	GetString GetChars
date [1] (SQL Server 2008 and later)	DateTime	Date [1]	GetSqlDateTime	Date [1]	GetDateTime
datetime	DateTime	DateTime	GetSqlDateTime	DateTime	GetDateTime
datetime2 (SQL Server 2008 and later)	DateTime	DateTime2	None	DateTime2	GetDateTime
datetimeoffset (SQL Server 2008 and later)	DateTimeOffset	DateTimeOffset	none	DateTimeOffset	GetDateTimeOffset
decimal	Decimal	Decimal	GetSqlDecimal	Decimal	GetDecimal
FILESTREAM attribute (varbinary(max))	Byte[]	VarBinary	GetSqlBytes	Binary	GetBytes
float	Double	Float	GetSqlDouble	Double	GetDouble
image	Byte[]	Binary	GetSqlBinary	Binary	GetBytes
int	Int32	Int	GetSqlInt32	Int32	GetInt32
money	Decimal	Money	GetSqlMoney	Decimal	GetDecimal
nchar	String	NChar	GetSqlString	StringFixedLength	GetString

Typ databázového stroje SQL Server	Typ rozhraní .NET Framework	Výčet SqlDbType	SqlDataReader SqlTypes typ přístupového objektu	Výčet DbType	Přístupový objekt typu SqlDataReader DbType
ntext	String	NText	GetSqlString	String	GetString
	Char[]				GetChars
numeric	Decimal	Decimal	GetSqlDecimal	Decimal	GetDecimal
nvarchar	String	NVarChar	GetSqlString	String	GetString
	Char[]				GetChars
real	Single	Real	GetSqlSingle	Single	GetFloat
rowversion	Byte[]	Timestamp	GetSqlBinary	Binary	GetBytes
smalldatetime	DateTime	DateTime	GetSqlDateTime	DateTime	GetDateTime
smallint	Int16	SmallInt	GetSqlInt16	Int16	GetInt16
smallmoney	Decimal	SmallMoney	GetSqlMoney	Decimal	GetDecimal
sql_variant	Object [2]	Variant	GetSqlValue [2]	Object	GetValue [2]
text	String	Text	GetSqlString	String	GetString
	Char[]				GetChars
time	TimeSpan	Time	none	Time	GetDateTime
(SQL Server 2008 and later)					
timestamp	Byte[]	Timestamp	GetSqlBinary	Binary	GetBytes
tinyint	Byte	TinyInt	GetSqlByte	Byte	GetByte
uniqueidentifier	Guid	UniqueIdentifier	GetSqlGuid	Guid	GetGuid
varbinary	Byte[]	VarBinary	GetSqlBinary	Binary	GetBytes
varchar	String	VarChar	GetSqlString	AnsiString, String	GetString
	Char[]				GetChars
varbinary	Byte[]	VarBinary	GetSqlBinary	Binary	GetBytes
varchar	String	VarChar	GetSqlString	AnsiString, String	GetString
	Char[]				GetChars
xml	Xml	Xml	GetSqlXml	Xml	none

SQL Server data type	CLR data type (SQL Server)	CLR data type (.NET Framework)
varbinary	SqlBytes, SqlBinary	Byte[]
binary	SqlBytes, SqlBinary	Byte[]
varbinary(1), binary(1)	SqlBytes, SqlBinary	byte, Byte[]
image	None	None
varchar	None	None
char	None	None
nvarchar(1), nchar(1)	SqlChars, SqlString	Char, String, Char[]
nvarchar	SqlChars, SqlString	String, Char[]
nchar	SqlChars, SqlString	String, Char[]
text	None	None
ntext	None	None
uniqueidentifier	SqlGuid	Guid
rowversion	None	Byte[]
bit	SqlBoolean	Boolean
tinyint	SqlByte	Byte
smallint	SqlInt16	Int16
int	SqlInt32	Int32
bigint	SqlInt64	Int64
smallmoney	SqlMoney	Decimal
money	SqlMoney	Decimal
numeric	SqlDecimal	Decimal
decimal	SqlDecimal	Decimal
real	SqlSingle	Single
float	SqlDouble	Double
smalldatetime	SqlDateTime	DateTime
datetime	SqlDateTime	DateTime
sql_variant	None	Object
User-defined type(UDT)	None	user-defined type
table	None	None
cursor	None	None
timestamp	None	None
xml	SqlXml	None